

IoT Pen Testing Report: Panamalar Door Sensor

Sulaiman Achmadi
sia373@student.bham.ac.uk

Nugrahani Cendekianingrum
nxc355@student.bham.ac.uk

Muhammad Khan
mak379@student.bham.ac.uk

Cédric Lüchinger
cxl1109@student.bham.ac.uk

Faheem Rahman
fxr375@student.bham.ac.uk

March 2024

University of Birmingham

Contents

1	Executive Summary	3
2	High-Level Device Description	4
3	Device Investigation	5
3.1	Analysing the device setup	5
3.1.1	EZ Mode	5
3.1.2	AP Mode	12
3.2	Analysing the device in use	14
3.3	Analysing the apk file	19
4	Attacks against the device	21
4.1	Attacks by a local attacker with Wi-Fi access	21
4.1.1	Man-in-the-Middle attack attempt	21
4.1.2	DDoS Attack	23
4.2	Attacks by a local attacker without Wi-Fi access	25
4.3	Attacks by a remote attacker	26
5	Analysis of weakness found	27
6	Appendix	28
7	References	29

1 Executive Summary

The pen test conducted on the Panamalar Door Window Sensor revealed that the IoT device and the phone with the “SmartLife” app always communicate through the cloud server of Tuya Smart with each other. The general encryption of the communicated data is well made. Nevertheless, the IoT device and the app are not fully secure. Due to the nature of UDP, which makes it easy to do DDOS attacks. Additionally, one can attach a magnet to the IoT device and thus disables the window door sensor to detect if it is open, this is a physical attack but nevertheless works and creates a vulnerability.

On the phone side, the app “SmartLife” opens potentially dangerous ports, especially MDNS port 5353, which leaves it open to exploits. Ultimately resulting in the device being relatively safe.

2 High-Level Device Description

This penetration testing report is centred around the Panamalar Door Window Sensor. The device is an alarm sensor that can maintain security around the user's home by sending notifications to the user when a door it is open or closed. This product is intended to benefit the user, by allowing them to check, from their smartphone, whether a door is open or closed, without having the user physically check the door.

The Panamalar Door Window Sensor is a device that can support smartphones running on Android 4.4 or iOS 8.0 and above and requires two AAA (triple A) batteries to operate. The connectivity is a 2.4 GHz Wi-Fi connection to the smartphone. The device comes with a magnetic device, which, when this is physically connected to the main door sensor, will send the user a notification stating that the door is closed.

To operate the device, the user will need to download the "SmartLife" application on their smartphone. This app is available on both the official App Store and Google Play. On the "SmartLife" app, the user can initially configure the device with their smartphone, something which requires a Bluetooth connection to complete. Once this is completed, the device will be connected to the user's smartphone via Wi-Fi, allowing them to have full access and control of the device.

3 Device Investigation

Before getting into the device setup, we want to talk about the IoT device itself and see if we already can find something with the information we have up until now. To get a closer investigation of the window door sensor of Panamalar and gain as much technical information as possible and as early as possible we want to do an FCC ID search. Since we can clearly see the FCC logo on the backside of the packaging, nevertheless we could not find an FCC identification. There is no FCC ID on the packaging, neither on the manual nor on the IoT Device itself nor online, even under the batteries there is no FCC ID. Since we do not have any further information, we conduct the FCC ID search via the company names we have given from the backside of the packaging. From the research with the Applicant Name Panamalar we cannot find anything. We further conducted search with the names AMZLAB, SeaMew Accounting ltd and shenzhen forever young technology co. ltd no match was found. But then when omitting the co. Ltd from the aforementioned company we find lots of different results, to be exact 60962 displaying records, starting from 1993 as the final action Date. Due to this fact and the limited time we have we cancelled any further search for the FCC ID of the window door sensor from Panalamar.

On a side note, we found that the official website of Panamalar uses http instead of https, which does not raise our hopes that the window door sensor of Panamalar is secure. In addition, none of the listed companies on the packaging have a website or at least their websites cannot be found by a regular search. Although one can find that AMZLAB GmbH based in Germany made their balance sheet of 2019 and 2020 in the year 2023. They did also have a managing director replacement on the 23.02.2022.

3.1 Analysing the device setup

The App, namely “Smart Life” has two distinct pairing modes, one via Wi-Fi the so-called AP Mode and another via Bluetooth the so-called EZ Mode. Due to this fact we split this sector into two different subsections.

3.1.1 EZ Mode

The EZ Mode is the desired mode regarding the manual of the window door sensor. Here is how to pair the devices. When starting up the App of the rooted phone, one can press the blue add button in the right corner. Then you can either connect manually via the selection of “Sensor” and then “Contact Sensor (WIFI)” After that you must type our Wi-Fi credentials of the network you would like to connect. In the next step one must open the battery cover and press the small black button until the light in front of the device is fast blinking. Now you have connected the phone successfully with your device.

A side from that you can also connect with the window door sensor from Panalamar automatically. For that just press the black button behind the battery cover and the device detects automatically your device once you have pressed the blue plus button in the app. You then just must press add and enter the Wi-Fi credentials and you are connected. But let's look what is happening underneath. The whole setup process on EZ Mode happens via Bluetooth.

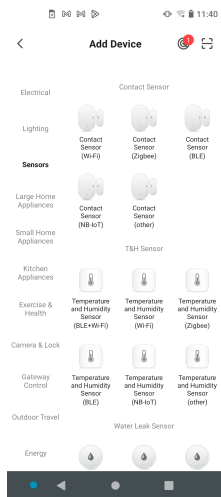


Figure 1: SmartLife device selection

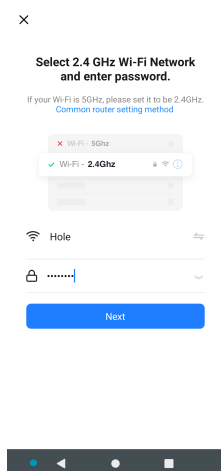


Figure 2: SmartLife Wi-Fi connection screen

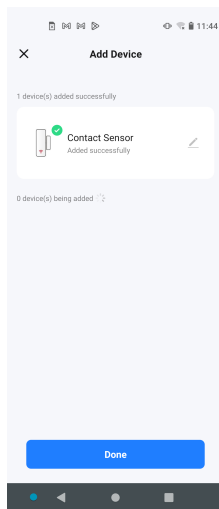


Figure 3: SmartLife screen upon successfully connection of a device



Figure 4: SmartLife showing the device as being connected



Figure 5: SmartLife showing the device as being disconnected

In order to perform this, we have installed the applications “nRF Connect” and “nRF Logger” to see what is happening on the Bluetooth level once we press the black button behind the battery cover since we were not able to spot initially any Bluetooth activity from the IoT device. But once we pressed the said button, we were able to detect a new Bluetooth device with the name “YT” which could correspond to the name of the company “Shenzhen Forever Young Technology Co. Ltd” namely a short name for “Young Technology”. Up on connection, we can take a view at the client and server side of the IoT devices Bluetooth. In Figure 1, one can see the server side which represents the IoT device. As one can see the generic attribute can notify changes to the client with the property” INDICATE”. On the contrary, the generic access can display two different kinds of information with their” READ” properties. So, one can only read from the on their own initiative.

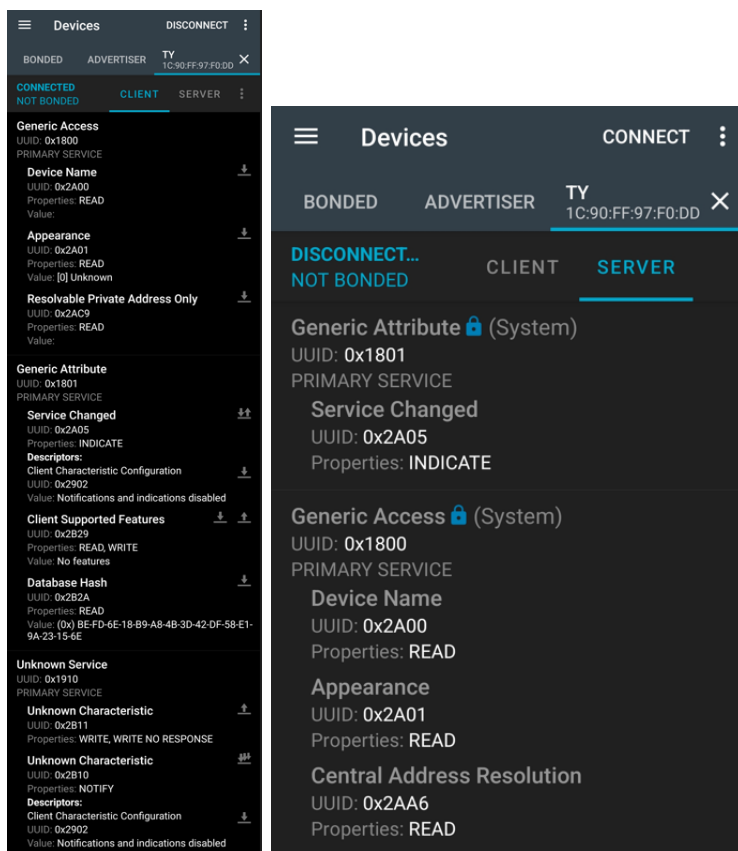


Figure 6 and 7: Client and Server side representing the IoT device

Moving now to the client side, we have found there the following insights. When taking a closer look at the image on the right one can see that we can read the device name due to the property “READ” although we are not able to receive any value. In a similar way we can read the appearance characteristic value, but this time we read there an unknown value, meaning it is either not specified or unidentified. The same as the device name applies to the characteristic resolvable private address only as one can see in the depicted image. Now moving to the generic attributes, the probably most interesting part here is the Database Hash which we can read the value, namely “(0x)BE-FD-6E-1B-B9-A8-4B-3D-42-DF-58-E1-9A-23-15-6E”. Then there is also an additional unknown service with some unknown characteristics, indicating that they might not be documented.

Now let's move to the most interesting part of the startup in EZ Mode. On how the devices communicate with each other. For that we enabled the HCI logging on the rooted phone. This functionality one can easily find in the developer options, one can see this option once having pressed seven times on the build number. Then we connected the rooted phone and our laptop to our own network. We must use our own network in order to not see something we are not intended to see. Additionally, we started up Wireshark and bettercap on our Kali Linux virtual machine to be able to view the protocols and what is when going on, on the network. Since we could not find anything useful in the first Wireshark runs. Thus, we set our network interface into promiscuous mode, in order to see all traffic on our network. Before that the NIC (network interface controller) processes only packets that are explicitly addressed to its own hardware MAC address or are broadcast addresses. Thus, all other packets will be ignored. In the resulting logs of Wireshark using promiscuous mode one can see the initialization and the startup of the IoT device in the network. Starting from the source address 0.0.0.0 which is a device which yet not has an IP address. In our case this is the Window Door Sensor from Panamalar. Initially the IoT device sends a DHCP (Dynamic Host Configuration Protocol) discover message to the broadcast address to indicate that the device wants to obtain an IP address. This is shown in Figure 8.

0.0.0.0	255.255.255.255	DHCP	352 DHCP Discover - Transaction ID 0x3f8e3539
---------	-----------------	------	---

Figure 8: DHCP discover message on Wireshark

Upon the response to the discover message, the window door sensor sends a request message using the same protocol and broadcast address. To receive its IP address. Shortly after that, as one can see below, the Tuya Smart cloud service is making an announcement that it has the IP address 192.168.75.14, using its MAC address. Funny enough the IP address of the cloud service is the same as the IP of the IoT device.

0.0.0.0	255.255.255.255	DHCP	352 DHCP Request - Transaction ID 0x3f8e3539
TuyaSmart_97:f0:dc		ARP	62 ARP Announcement for 192.168.75.14

Figure 9: Wireshark capture of cloud IP address

In a next step, since Tuya Smart has not cached the MAC address cached of the gateway it asks for it to be able to communicate properly.

19121 140.354113791 TuyaSmart_97:f0:dc	ARP	62 Who has 192.168.75.41? Tell 192.168.75.14
--	-----	--

Figure 10: Wireshark capture of changed gateway

After that the communication is fully established and the devices start communicating. This was the Wi-Fi part of the analysis of the communication. Now we must investigate what happened on the Bluetooth level.

A bluetooth traffic analysis was done using the built-in device logs, stack traces, and other diagnostic information available from the bug report. From the report file, we can access the bluetooth logs through /FS/data/misc/bluetooth/logs. There will be two log files: named btsnoop.hci.log and btsnoop.hci.log.last, with the latter having a longer recorded traffic packets. Accessing the logs file gave us information about what types of bluetooth protocols are used during the device's synchronization with the IoT device.

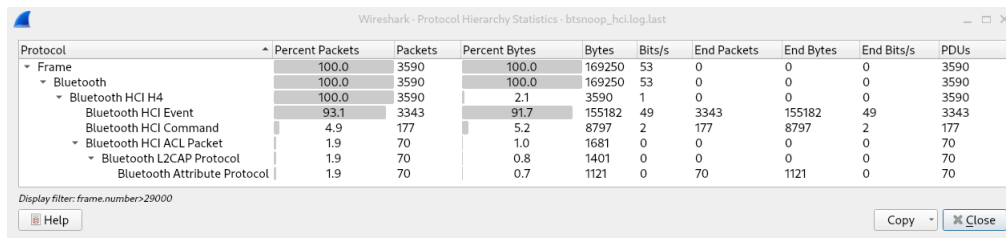


Figure 11: Bluetooth traffic analysis

From the protocol hierarchy, Bluetooth HCI Event was the most common, followed by Bluetooth HCI Command. However, both protocols are used only in communications between the host (Android’s microcontroller unit) and the controller (bluetooth chipset). Meanwhile, the only protocol that communicates outside the device is the Bluetooth Attribute Protocol (ATT). The ATT protocols were the protocol used to communicate between the rooted Android phone (Motorola One Vision) and the IoT Device (TuyaS-smart). Tuya is a third-party application developer of the Panamalar Smart Life Android APK.

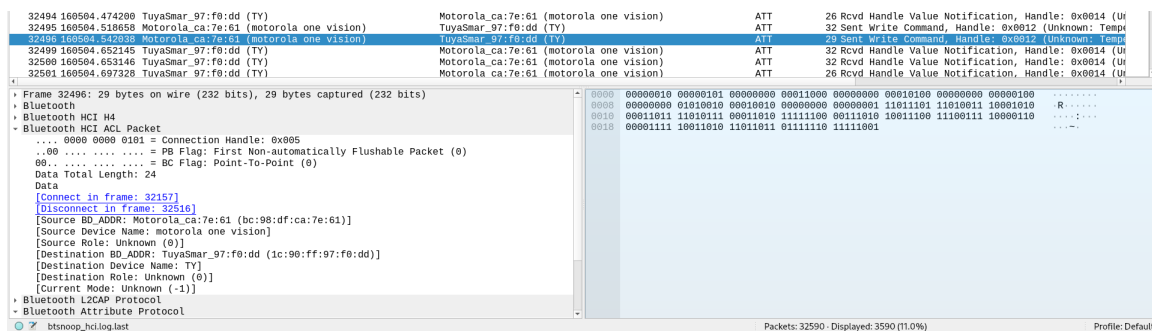


Figure 12: Wireshark capture showing the ATT protocol

Unfortunately, none of the packets sent and received by the IoT device seemed to contain anything useful. All 70 of the ATT packets were short in size (less than 32 bytes) and does not contain any decryptable values. Similar information was found during packet inspection using Bettercap’s Bluetooth Low Energy devices discovery module. Since we have not found anything promising on the Bluetooth analysis. We tried another approach to maybe gain some additional information. As earlier stated, we used “nRF connect” and “nRF logger” on our private android phones (“nRF logger” does not exist on the apple store). For that reason, we connected our private phones with the IoT device and the rooted phone on Bluetooth. Then we carry out again the start-up process. But the result left much to be desired. We gained no additional knowledge.

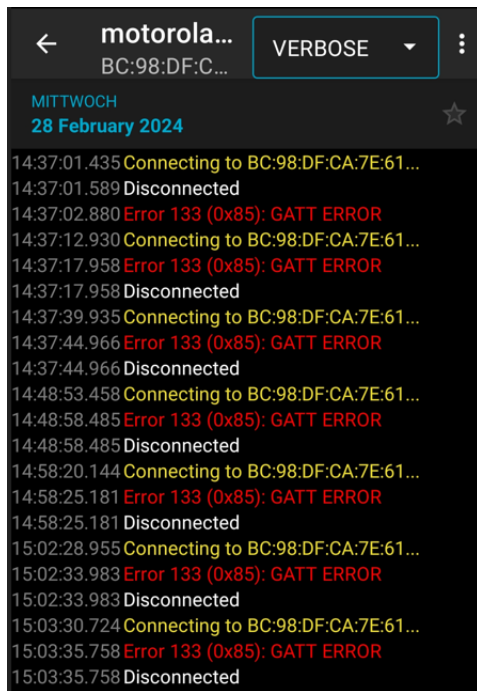


Figure 13: Wireshark capture of log in "verbose mode"

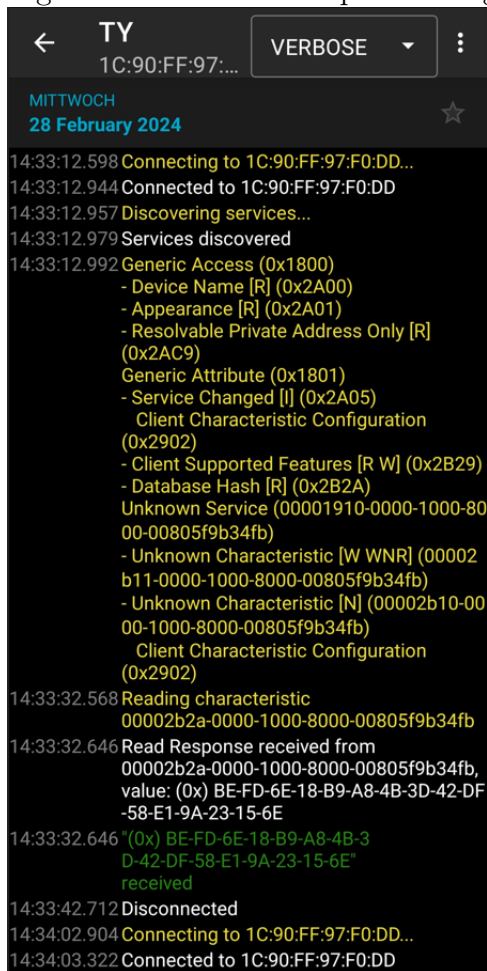


Figure 14: Wireshark capture of log in "verbose mode"

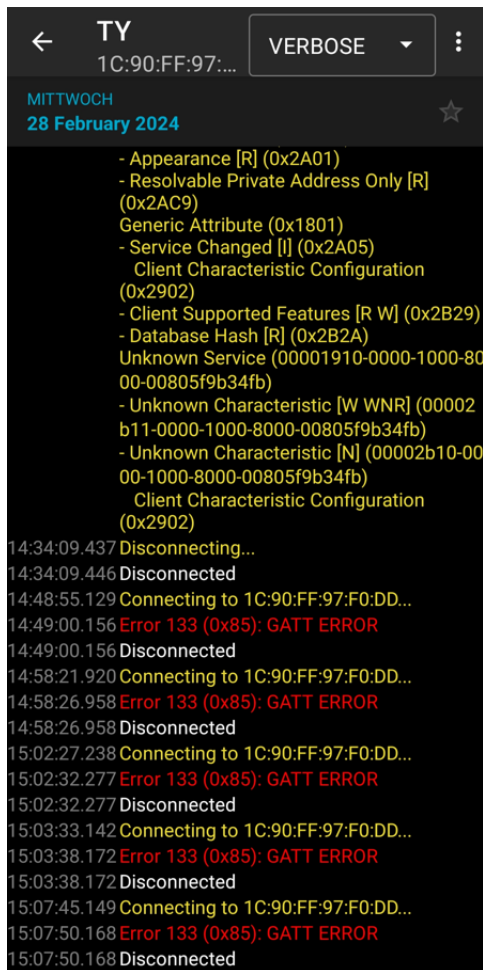


Figure 15: Wireshark capture of log in "verbose mode"

3.1.2 AP Mode

According to the manual of the Panamalar window door sensor, the AP Mode is the mode which you must use as a backup when you were previously not able to connect with the IoT device. To activate this mode, one must press the black button behind the battery cover so long that the window door sensor starts blinking slowly. Then you can see a Wi-Fi appears with the name SmartLife-F0DC and has no password. Then you connect the rooted phone with the Smart Life app installed with the Wi-Fi. Initially once one enters the SmartLife-App on the rooted phone it starts immediately scanning the network for IoT devices. Which one can see with the following packet displayed in the network; it has its destination 224.0.0.251:5353 using the MDNS (Multicast DNS) protocol.



Figure 16: Wireshark capture MDNS protocol

At the first glance this might seem a little bit strange, but on further investigation it is common practise to detect new devices with zero or nearly zero configuration. In our case this is clearly done for detection of the device. Due to the fact we want to know

what happens when one is connecting to the Wi-Fi. We log ourselves into the network of the IoT device with Wireshark active. First thing we see is that TuyaSmart_97:f0:dd answers to our ARP request with being said that it is the gateway for us.

Intel_e3:b3:c2	Broadcast	ARP	60 Who has 192.168.175.1? Tell 192.168.175.100
TuyaSmart_97:f0:dd	Intel_e3:b3:c2	ARP	60 192.168.175.1 is at 1c:90:ff:97:f0:dd

Figure 17: Wireshark capture of ARP request

Later we can spot two interesting messages in the log with SOAP (Simple Object Access Protocol) data in them as plain text. The first one is from a IPv6 address and requests a specific UUID.

fe80::87b:a69a:9f59:1568	ff02::c	UDP	718 56303 → 3702 Len=656
--------------------------	---------	-----	--------------------------

Figure 18: Wireshark capture of log with SOAP data

The second which is again used for a service discovery but with IPv4. Due to the fact that the message comes from the same devices, just one in IPv4 and the other message in IPv6 they have different UUID displayed, but ultimately associated with the same service.

192.168.175.100	239.255.255.250	UDP	698 56302 → 3702 Len=656
-----------------	-----------------	-----	--------------------------

Figure 19: Wireshark capture of different UUID

Apart from that there is no additional important information to find, because we were not able to directly monitor the traffic on the SmartLife-F0DC Wi-Fi when the phone enters the said network, since when it detects that there are more than two devices connected to the Wi-Fi it will turn itself off. Due to this fact, we were not able to catch useful data regarding the AP startup mode. Another way we found to detect the data traffic is using airodump-ng from aircrack-ng. In order to succeed, we used an external USB Wi-Fi adapter, else we are not able to detect that kind of Wi-Fi traffic, due to the limitations of our laptops. In the next step, we set everything up, installed the driver for the adapter, ready to analyse. Sadly, due to incompatibility issues of the USB Wi-Fi adapter, we were not able to detect any Wi-Fi near our location. Below one can see the setup.

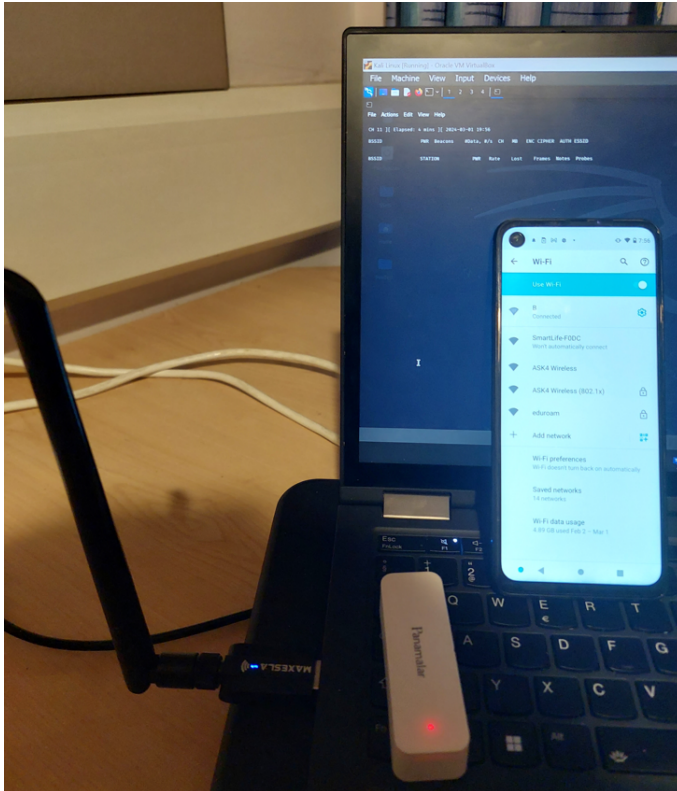


Figure 20: Device setup

3.2 Analysing the device in use

There is no difference with the communication whether the IoT device and rooted phone are in a different network or in the same one. In the beginning we must detect which devices are on our network and which IP address do they have, to be later able to filter after specific devices. Since the communication else can be quite crowded and hard to analyse. For detection what kind of devices are connected to the network. We used the command “net.probe on” to do that. In addition to that we used the command “net.show” to have a more structured overview. One can see the result in the image below. On the other hand, one could do the same with the with opening the terminal and the command “sudo nmap -sn 192.168.61.0/24” on linux (depending on the IP address range). The “-sn” indicates that we are doing a ping scan and the “/24” indicates the subnet mask which further denotes that the scan should cover all IP addresses in that range instead of a specific IP address.

```

192.168.61.0/24 > 192.168.61.210 » net.probe on
192.168.61.0/24 > 192.168.61.210 » [14:25:53] [sys.log] [inf] net.probe starting net.recon as a requirement for net
.net.probe
192.168.61.0/24 > 192.168.61.210 » [14:25:53] [sys.log] [inf] net.probe probing 256 addresses on 192.168.61.0/24
192.168.61.0/24 > 192.168.61.210 » [14:25:54] [endpoint.new] endpoint 192.168.61.57 detected as 5a:bc:1c:c4:23:20.
192.168.61.0/24 > 192.168.61.210 » [14:25:55] [endpoint.new] endpoint 192.168.61.135 detected as 78:af:08:e3:b3:c2.
192.168.61.0/24 > 192.168.61.210 » net.show

```

IP	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.61.210	08:00:27:74:e1:4e	eth0	PCS Computer Systems GmbH	0 B	0 B	14:21:50
192.168.61.221	66:5f:c9:cb:ef:46	gateway		3.6 kB	6.9 kB	14:21:50
192.168.61.57	5a:bc:1c:c4:23:20			15 kB	552 B	14:26:34
192.168.61.135	78:af:08:e3:b3:c2			0 B	460 B	14:25:55

Figure 21: Bettercap capture of IP address

From the information we see, and we see that we have 4 different kind of IP addresses on our network. As we see on our terminal, we have the IP address ending of 210 on our virtual machine. Further we can spot the gateway with the ending 221, in our test environment this is the hotspot phone. Then there are two other different devices. To find out which device is our rooted phone, we can easily go into the Wi-Fi settings and there find the rooted phones IP address, which has in our case the ending 57. This leaves us the ending 135 for the laptop. One can easily confirm this assumption in the hotspot settings of the hotspot phone. Now we start the logging on Wireshark, activate Bluetooth and locations on the rooted phone. Speaking of Bluetooth, since it has no more effect on the communication we will omit it in further discussions. Now let's start working. After connecting the phone to the window door sensor from Panalamar one can see three different kinds of messages reoccurring on Wireshark. First we look at the packets that contain the IP address of the rooted phone for that, we type into the filter of Wireshark "ip.addr==192.168.61.57". Now only results concerning the android phone are going to be displayed. Let's take a closer look at the log created during use. There are always the same two messages reoccurring. Due to this fact we are going to analyse each of them. The first occurring message uses the UDP protocol to the destination 255.255.255.255:7000. One can see an image of the message below.

```
192.168.61.57 255.255.255.255 UDP 127 33267 → 7000 Len=85
```

Figure 22: Wireshark capture of UDP protocol

The info part displays the source port namely 33267 sent to every device on the local network to their port 7000. Since the destination address is 255.255.255.255 which is used for broadcasting. This means that it is possible that every other device within the network will receive this message on the port 7000. As earlier mentioned, the port 7000 is used for cloud server which is in our case Tuya Smart and thus being ultimately sent to them. Luckily, the data sent is encrypted, as one can see on the picture below. The blue highlighted part is the data part of the message sent. With their checksums "0x91ca" reading just before the highlighted part.

```
0000 ff ff ff ff ff ff 5a bc 1c c4 23 20 08 00 45 00 .....Z..#..E.
0010 00 71 77 81 40 00 40 11 c5 19 c0 a8 3d 39 ff ff ..qw.@.@...=9..
0020 ff ff 81 f3 1b 58 00 5d 91 ca 00 00 66 99 00 00 .....X.]...f...
0030 00 00 00 00 00 00 25 00 00 00 3f 51 d4 60 8e .....%...?Q...
0040 78 34 11 23 f6 b0 47 c9 9c 38 d5 56 95 a6 cb 95 x4.#..G..8.V...
0050 4b 63 4d 67 1b 27 49 f6 40 ad 0f 78 6b 06 c7 6d KcMg.'I..@..xk..m
0060 52 63 31 d2 8c 7a 1c c0 e2 1d 52 2c 59 9d e7 d7 Rc1..z...R,Y...
0070 a0 56 81 f3 80 42 e0 19 e4 4d 2d 00 00 99 66 ..V...B...M...f
```

Figure 23: Bettercap capture of encrypted data

Moving now to the third and last type of packet that is being sent from the rooted phone, shown in Figure 23.

```
192.168.61.57 192.168.61.255 UDP 127 47209 → 7000 Len=85
```

Figure 24: Bettercap capture of encrypted data

This packet has a similar use as the previous shown packet. They are both broadcast addresses send to the same port. A reason for this is that they want to ensure that the packets really get to their destination. Let's now take a closer look at the Wireshark entries from the IoT device, displayed underneath. We can see that it is a UDP

broadcast packet being sent to port 6667. But the source port always alters when being sent. This is a typical mechanism of port randomization, which is used for mitigation of port exhaustion, to enhance the security and to improve the load balancing. Thus, ultimately the packet is being received again at the Tyua Smart cloud server, wich leads us to the visualisation from the Window Door Sensor of Panamalar to the phone with the “SmartLife” App on it. Both devices communicate with each other over the cloud server.

192.168.61.14 255.255.255.255 UDP 246 53743 → 6667 Len=204

Figure 25: Wireshark entries of IoT device

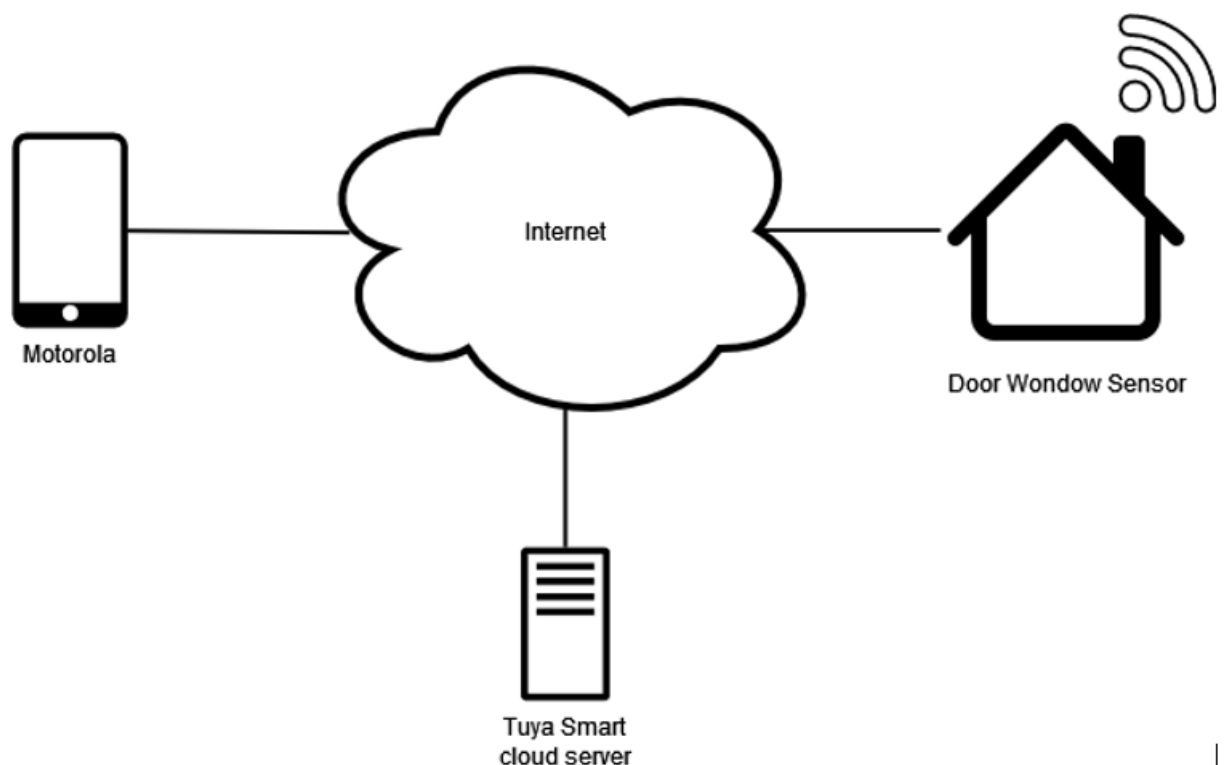


Figure 26: System overview diagram

Funny enough, when looking at the bettercap terminal one can see that the Panalar window door sensor (IP address 192.168.61.14) is always visible on the network for a moment once it gets activated due to manual manipulation of the IoT device. Namely opening and closing the door. But then the endpoint gets lost.

```

192.168.61.0/24 > 192.168.61.210 » [14:39:56] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [14:40:06] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [14:40:15] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [14:40:34] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [14:40:40] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [14:40:58] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [14:41:07] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
  
```

Figure 27: Bettercap capture of endpoint

This is done for energy saving and security reasons. Since, if someone would do for example a nmap scan the device is not visible. We have done such a nmap scan and we were not able to find the IoT device, as previously stated. The scan below is performed on a different time but the IP address of the window door sensor has still the ending .14.

```
(kali)-[~]
$ sudo nmap -sn 192.168.24.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-03-02 14:47 GMT
Nmap scan report for 192.168.24.135
Host is up (0.00037s latency).
MAC Address: 78:AF:08:E3:B3:C2 (Intel Corporate)
Nmap scan report for 192.168.24.185
Host is up (0.29s latency).
MAC Address: AA:6D:BA:6A:CB:B3 (Unknown)
Nmap scan report for 192.168.24.193
Host is up (0.16s latency).
MAC Address: 0A:BC:DC:2B:ED:73 (Unknown)
Nmap scan report for 192.168.24.210
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.66 seconds
```

Figure 28: nmap scan unsuccessfully showing the device

Up on the ping scan of the network, we performed an aggressive (-A) scan of all UDP (-sU) ports, without trying to determine if the host is available (-Pn). Since we already know that the host is available and that it uses UDP. We do an aggressive scan to include OS detection. Only after doing a nmap scan of the IoT device with skipping the host discovery, perfect timing that the window door sensor is online and keeping it online through constant open and closing of the sensor, one can conduct an Nmap scan. We can do that since we know what the IP address is from the IoT device due to the discoveries in bettercap.

```
Nmap scan report for 192.168.24.14
Host is up (0.18s latency).
All 1000 scanned ports on 192.168.24.14 are in ignored states.
Not shown: 861 closed udp ports (port-unreach), 139 open/filtered udp ports (no-response)
MAC Address: 1C:90:FF:97:F0:DC (Tuya Smart)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: 2N Helios IP VoIP doorbell (95%), Advanced Illumination DCS-100E lighting controller (95%), AudioControl D3400 network amplifier (95%), British Gas GS-Z3 data logger (95%), Daysequerra M4.2SI radio (95%), Denver Electronics AC-5000W MK2 camera (95%), DTE Energy Bridge (lwIP stack) (95%), Enlogic PDU (FreeRTOS/lwIP) (95%), Espressif esp8266 firmware (lwIP stack) (95%), Espressif ESP8266 WiFi system-on-a-chip (95%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 183.09 ms 192.168.24.14

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 4478.12 seconds
```

Figure 29: nmap scan of IoT device

As one can see by the nmap report of the IoT device (192.168.24.14) above, which gave us the insight on which OSs are like the OS of our IoT device.

On the same go we performed also a port scan of the research phone.

```
Nmap scan report for 192.168.24.185
Host is up (0.064s latency).
Not shown: 994 closed udp ports (port-unreach)
PORT      STATE      SERVICE    VERSION
1035/udp   open|filtered mxxrlogin
5353/udp   open|filtered zeroconf
17638/udp  open|filtered unknown
21780/udp  open|filtered unknown
29810/udp  open|filtered unknown
51972/udp  open|filtered unknown
MAC Address: AA:6D:BA:6A:CB:B3 (Unknown)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1 64.00 ms 192.168.24.185
```

Figure 30: Port scan

The first service we detected was “mxxrlogin”, it is a potential vulnerability since it uses the MX-XR RPC protocol, which allow the programs to execute procedures or functions on remote systems.¹ The second service we discovered was the “zeroconf” service using the port 5353, this is a very common port used for discovery of devices on the same local network. But it is also a little worrying since a lot of different vulnerabilities can be found online.² For clarity, we initially only did every run in normal mode but we were not able to see anything useful, we only saw the UDP messages. To see all packets, we had to put it into promiscuous mode. Here is on how we did it. We opened the terminal and wrote the command “ifconfig” to be able to see, on which interface we are connected with the network. In our case it is eth0. In order to set it into promiscuous mode, we wrote the command “sudo ifconfig eth0 promisc”. And at the end we convince ourselves it’s in promiscuous mode. Which one can verify onto the below.

```
eth0: flags=4419<UP,BROADCAST,RUNNING,PROMISC,MULTICAST> mtu 1500
    inet 192.168.75.210 netmask 255.255.255.0 broadcast 192.168.75.255
    inet6 fe80::a00:27ff:fe74:e14e prefixlen 64 scopeid 0x20<link>
```

Figure 31: Promiscuous mode

3.3 Analysing the apk file

We were able to analyse the SmartLife APK file by decompiling the source code using JADX. From the resulting code, we were able to figure out that AES encryption is used here, and we were able to figure out the encryption key that is distributed via the Bluetooth which is (0x)BE-FD-6E-1B-B9-A8_4B-3D-42-DF-58-E1-9A-23-6E. We can see, in Figure 32, that AES encryption is used with CBC. AES is known to be quite a secure form of encryption. This figure also shows that PKCS5 padding is used. The key length that is used for the RSA key encryption is 2048, which is quite a large key size. The length of the secret key that is used in the encryption is 256 bits. The key used for the encryption what we have discovered above. RSA encryption is also used here as well with ECB encryption. ECB encryption is not as secure as CBC encryption because ECB is susceptible to error propagation. RSA decryption is possible, as shown further below.

```
/* Loaded from: classes.dex */
4 public final class AESEncryptor {
    private static final String AES = "AES";
    private static final String AES_CBC_PKCS7_PADDING = "AES/CBC/PKCS7Padding";
    private static final int AES_SECRET_KEY_LENGTH = 256;
    private static final String ANDROID_KEYSTORE = "AndroidKeyStore";
    public static final int INIT_VECTOR_LENGTH = 16;
    private static AESEncryptor INSTANCE = null;
    private static final String KEYSTORE_ALIAS = "LMA_KEYSTORE_ALIAS";
    private static final String RSA = "RSA";
    private static final String RSA_ECB_PKCS1_PADDING = "RSA/ECB/PKCS1Padding";
    private static final int RSA_ENCRYPTION_KEY_LENGTH = 2048;
    private static final String TAG = "AESEncryptor";
    private static final String VERSION_PREFIX = "AES_00";
    private final Context mContext;
    private final byte[] mEncryptionKey;
    private final SecretKeySpec mKeySpec;
    private KeyStore mKeyStore;
```

Figure 32: AES encryption

As mentioned before, this device uses RSA encryption. With the RSA encryption, comes the use of a key, the code for the key generation is shown here below.

```
18) generateKeysIfNeeded(Context context) throws KeyStoreException, InvalidAlgorithmParameterException, NoSuchAlgorithmException, NoSuchProviderException {
19     keyStore = this.mKeyStore;
20     if (keyStore != null) {
21         keyStore.containsAlias(KEYSTORE_ALIAS);
22         return;
23     }
24     return;
25     .eg.i(TAG, "Generating RSA keypair...");
26     Calendar calendar = Calendar.getInstance();
27     Calendar calendar2 = Calendar.getInstance();
28     int year = calendar2.get(1, 0);
29     int month = calendar2.get(2, 0);
30     int day = calendar2.get(3, 0);
31     KeyPairGeneratorSpec.Builder builder = new KeyPairGeneratorSpec.Builder(context).setAlias(KEYSTORE_ALIAS).setSubject(new X500Principal("CN=LMA_KEYSTORE_ALIAS")).setSerialNumber(BigInteger.TEN).setKeySize(2048).setStartDate(calendar.getTime()).setEndDate(calendar2.getTime()).build();
32     KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance(RSA, ANDROID_KEYSTORE);
33     keyPairGenerator.initialize(builder);
34     keyPairGenerator.generateKeyPair();
35     return;
36     throw new IllegalArgumentException("KeyStore is null");
```

Figure 33: RSA key generation

This device requires the use of a Bluetooth connection, and where the connection is made is shown below here.

```
public void establishPhaseConnection(long j, int i, long j2) {
    if (this.connectionId == 0) {
        this.connectionId = j;
        if (j == 0) {
            this.completionListener.onError(new Exception("Failed to add Bluetooth connection."));
            return;
        }
        Stringbuilder sb = new Stringbuilder();
        sb.append("Establishing phase connection with ID: ");
        sb.append(this.connectionId);
        Stringbuilder sb2 = new Stringbuilder();
        sb2.append("Establishing phase connection with ID: ");
        sb2.append(j);
        establishPhaseConnection(this.deviceControllerPtr, j, i, j2);
        return;
    }
    this.completionListener.onError(new Exception("Bluetooth connection already in use."));
}

public native byte[] extractSkidFromPaaCert(byte[] bArr);

public void finalize() throws Throwable {
    super.finalize();
    long j = this.deviceControllerPtr;
    if (j != 0) {
        deleteDeviceController(j);
        this.deviceControllerPtr = 0;
    }
}
```

Figure 34: Bluetooth connection which will use the key mentioned earlier

The decryption for the RSA has also been picked up here, shown below.

```
public static byte[] decrypt(String str, PrivateKey privateKey, byte[] bArr) throws Exception {
    try {
        if (TextUtils.isEmpty(str)) {
            str = "RSA";
        }
        Cipher cipher = Cipher.getInstance(str, "BC");
        cipher.init(2, privateKey);
        int blockSize = cipher.getBlockSize();
        ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream(64);
        int i = 0;
        while (true) {
            int i2 = i * blockSize;
            if (bArr.length - i2 > 0) {
                byteArrayOutputStream.write(cipher.doFinal(bArr, i2, blockSize));
                i++;
            } else {
                return byteArrayOutputStream.toByteArray();
            }
        }
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}
```

Figure 35: RSA decryption

The communication between the gateway and third-party applications is secured by TLS. Figure 36 shows the mention of TLS in the jadx file.

```
private Socket enableTLSOnSocket(Socket socket) {
    if (socket != null && (socket instanceof SSLSocket)) {
        ((SSLSocket) socket).setEnabledProtocols(new String[]{"TLSv1.2"});
    }
    return socket;
}
```

Figure 36: TLS

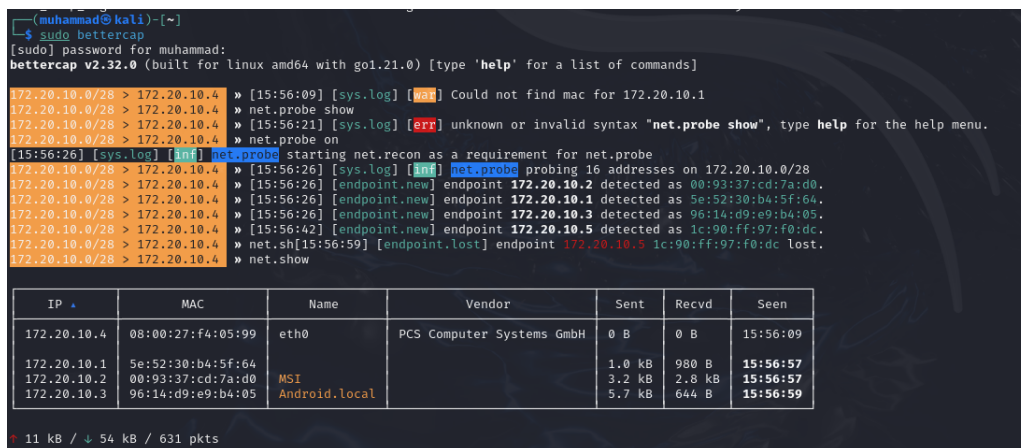
4 Attacks against the device

4.1 Attacks by a local attacker with Wi-Fi access

4.1.1 Man-in-the-Middle attack attempt

Being within the Wi-Fi range of the device. There are many types of attacks the attacker can perform. Man in the middle attack is when an attacker attempts to intercept communication between two parties without them knowing. In this case we would attempt to gain information being transmitted from the device to the window sensor and vice versa. Hopefully we would be able to intercept and potentially manipulate that information if successful.

To start with this, we would first need to download the relevant software. I chose to install bettercap on kali linux as from my research this is a viable software often used for this purpose. One installing we initiate better cap with the command “sudo bettercap”. Next, we write the command “net.probe” which then shows all the devices that are connected to the network. The phone has an IP address of 172.20.10.3 and the window sensor has the IP 172.20.10.5.



```
(muhammad@kali)~  
$ sudo bettercap  
[sudo] password for muhammad:  
bettercap v2.32.0 (built for linux amd64 with go1.21.0) [type 'help' for a list of commands]  
172.20.10.0/28 > 172.20.10.4 » [15:56:09] [sys.log] [warn] Could not find mac for 172.20.10.1  
172.20.10.0/28 > 172.20.10.4 » net.probe show  
172.20.10.0/28 > 172.20.10.4 » [15:56:21] [sys.log] [err] unknown or invalid syntax "net.probe show", type help for the help menu.  
172.20.10.0/28 > 172.20.10.4 » net.probe on  
[15:56:26] [sys.log] [info] net.probe starting net.recon as a requirement for net.probe  
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [sys.log] [info] net.probe probing 16 addresses on 172.20.10.0/28  
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.2 detected as 00:93:37:cd:7a:d0.  
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.1 detected as 5e:52:30:b4:5f:64.  
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.3 detected as 96:14:d9:e9:b4:05.  
172.20.10.0/28 > 172.20.10.4 » [15:56:42] [endpoint.new] endpoint 172.20.10.5 detected as 1c:90:ff:97:f0:dc.  
172.20.10.0/28 > 172.20.10.4 » net.sh[15:56:59] [endpoint.lost] endpoint 172.20.10.5 1c:90:ff:97:f0:dc lost.  
172.20.10.0/28 > 172.20.10.4 » net.show
```

IP	MAC	Name	Vendor	Sent	Recv	Seen
172.20.10.4	08:00:27:f4:05:99	eth0	PCS Computer Systems GmbH	0 B	0 B	15:56:09
172.20.10.1	5e:52:30:b4:5f:64	MSI		1.0 kB	980 B	15:56:57
172.20.10.2	00:93:37:cd:7a:d0			3.2 kB	2.8 kB	15:56:57
172.20.10.3	96:14:d9:e9:b4:05	Android.local		5.7 kB	644 B	15:56:59

11 kB / 54 kB / 631 pkts

Figure 37: Bettercap capture of phone and window sensor

```

(muhammad@kali)~$ sudo bettercap
[sudo] password for muhammad:
bettercap v2.32.0 (built for linux amd64 with go1.21.0) [type 'help' for a list of commands]

172.20.10.0/28 > 172.20.10.4 » [15:56:09] [sys.log] [warn] Could not find mac for 172.20.10.1
172.20.10.0/28 > 172.20.10.4 » net.probe show
172.20.10.0/28 > 172.20.10.4 » [15:56:21] [sys.log] [err] unknown or invalid syntax "net.probe show", type help for the help menu.
172.20.10.0/28 > 172.20.10.4 » net.probe on
[15:56:26] [sys.log] [info] net.probe starting net.recon as a requirement for net.probe
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [sys.log] [info] net.probe probing 16 addresses on 172.20.10.0/28
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.2 detected as 00:93:37:cd:7a:d0.
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.1 detected as 5e:52:30:b4:5f:64.
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.3 detected as 96:14:d9:e9:b4:05.
172.20.10.0/28 > 172.20.10.4 » [15:56:26] [endpoint.new] endpoint 172.20.10.5 detected as 1c:90:ff:9f:f0:dc.
172.20.10.0/28 > 172.20.10.4 » net.sh[15:56:59] [endpoint.lost] endpoint 172.20.10.5 1c:90:ff:9f:f0:dc lost.
172.20.10.0/28 > 172.20.10.4 » net.show

```

IP	MAC	Name	Vendor	Sent	Recv	Seen
172.20.10.4	08:00:27:f4:05:99	eth0	PCS Computer Systems GmbH	0 B	0 B	15:56:09
172.20.10.1	5e:52:30:b4:5f:64			1.0 kB	980 B	15:56:57
172.20.10.2	00:93:37:cd:7a:d0	MSI		3.2 kB	2.8 kB	15:56:57
172.20.10.3	96:14:d9:e9:b4:05	Android.local		5.7 kB	644 B	15:56:59

11 kb / 54 kb / 631 pkts

Next, we write the command “set.arp.spoof.duplex true” as bettercap performs ARP spoofing in full duplex mode so data can be sent and received simultaneously. Then we specify who we are trying to spoof so for this we chose to attack the phone, so we enter the command “set arp.spoof.targets 172.20.10.3”. To start the spoofing, we then entered the command “arp.spoof on”. Doing this we have initiated ARP spoofing with bettercap. Next, we use the command “net.sniff.on” to analyse what’s being transmitted from the device. This command displays the source and destinations of the IP addresses, and the protocols used.

[illegible]

22

With packet sniffing with bettercap via Linux. With the command “net.sniff on” we were able to capture and analyse network traffic on this network interface as the IOT device was connected to the same network. We kept attaching and detaching the sensor to analyse what was being sent. Not much was shown. This is positive in the defence of the device as it prevents the hacker from seeing much.

4.1.2 DDoS Attack

```

(muhammad@kali:~)[-]
$ cd Desktop
(muhammad@kali:~/Desktop)
$ mkdir scapy
(muhammad@kali:~/Desktop)
$ cd scapy
(muhammad@kali:~/Desktop/scapy)
$ git clone https://github.com/secdev/scapy
Cloning into 'scapy'...
remote: Enumerating objects: 40827, done.
remote: Counting objects: 100% (40827/40827), done.
remote: Compressing objects: 100% (12039/12039), done.
remote: Total 40827 (delta 28165), reused 40514 (delta 28040), pack-reused 0
Receiving objects: 100% (40827/40827), 82.92 MiB | 883.00 KiB/s, done.
Resolving deltas: 100% (28165/28165), done.
(muhammad@kali:~/Desktop/scapy)
$ ls
scapy
(muhammad@kali:~/Desktop/scapy)
$ cd scapy
(muhammad@kali:~/Desktop/scapy/scapy)
$ ls
CONTRIBUTING.md LICENSE MANIFEST.in README.md doc pyproject.toml run_scapy run_scapy.bat scapy setup.py test tox.ini
(muhammad@kali:~/Desktop/scapy/scapy)
$ /run_scapy
(NFO: Can't import PyX. Won't be able to use psump() or pdfdump()).

      aSPV//VAsa
      aPPpPYCY//II//NCA
      sV//II//SPscs spcY/Pp | Welcome to Scapy
      ayp aPPpPYCY//II//NCA | Version 2.5.0.dev295
      AVAsA//////////Bs spY/C//s | https://github.com/secdev/scapy
      pCCCY//I/ cSPss y/I/ |
      sPPp//I/A cyP//II/AC//Y |
      A/I/A cyP//II/AC//Y | Have fun!
      p//II/AC p//II/AC//Y |
      P//II/PCpc A/I/A | Craft packets before they craft
      scccccc//gsp//I/p p/I/y | you.
      sV//II//I/y caa s/I/p -- Socrate
      caaCryp//I/a vY/V/
      sV/PsV//II/NCC ac//I/p
      sc sccaCV//PCYpaayC//Yss
      spcPV//II//VPPps
      ccaacs

using IPython 8.20.0

>>> conf
ASN1_default_codec = <ASN1Codec BER[1]>
ASN1_codec_size = 0
AS_resolver = <scapy.as_resolvers.AS_resolver_multi object at 0x7f628e7ab90>
BTSocket = <BTBluetoothRFCOMMSocket: read/write packets on a connected L2CAP...
LL2listen = <LL2listenSocket: read packets at layer 2 using Linux PF_PACKET ...
LL3socket = <LL3socket: read/write packets at layer 2 using Linux PF_PACKET ...
LL3socket = <LL3PacketSocket: read/write packets at layer 3 using Linux PF_P...
LL3socket = <FuncTools.partial(<LL3PacketSocket: read/write packets at layer ...
auto_crop_tables = True
auto_fragment = True
bufsize = 65536
cache.scapy[1]set = {}

```

We ran into an error when trying to run code in the normal terminal due to permissions. We overcome this when we ran the same code and ran scapy in the root terminal.



Figure 43: DDoS working. Note the fact that the sensor is not physically connected to the device despite the phone saying it is connected

4.2 Attacks by a local attacker without Wi-Fi access

Since the communication of the window door sensor with the phone goes only through the cloud server, it needs internet access. Nevertheless, there are a few possibilities on to attack the IoT device without Wi-Fi access. One possibility is to use airodump-ng as we did earlier in the AP Mode part, listen to the communication and just block every communication going to the cloud server Tuya Smart. The only way it prevents from being listen from the outside is the Wi-Fi password and the encryption of the data. Apart from that one can always guess the Wi-Fi password or brute force it.

Not to forget a local attacker might not only use informatics to attack, one can also use physical attacks. For that we found out that one can use a magnet to keep the state of the IoT device closed. If you add the magnet fast enough, then the state of the device does not change at all. Thus, also being a valid and easy way to rule out the functionality of the window door sensor. There is no active protection against this, in fact it is only supported by the communication through the slow cloud server communication and their time delay. In addition to that one could also mimic with “nRF connect” the window door sensor and trick the user into connection with their device instead of the IoT device. Although it is hard to reverse engineer the IoT device via Bluetooth and one must be near the location at the exact moment when the user has decided to set his IoT device up. Due to the fact that it is challenging and the time window of doing such an attack is very small and random, the likelihood is very small.

4.3 Attacks by a remote attacker

One thing that a remote attacker could do is probing with the found MAC address of Tuya Smart (TuyaSmart_97:f0:dd) by just changing the last two digits in order to see if he receives a response from then. Other than that, and a man in the middle attack (MITM), these are the only attacks that an attacker could do due to the strong encryption present.

An example attack which cannot be carried out by the attack is eavesdropping. This involves passively capturing data transmitted over the Internet to steal sensitive information like credentials or personal data. Without the use of strong encryption, data is vulnerable to interception, but strong encryption protocols like TLS (Transport Layer Security) can render intercepted data unreadable to unauthorised parties, which the jadx file of the app's source code does reveal, but since our device uses AES-CBC encryption, this makes the device less likely to be susceptible for eavesdropping attacks.

Another type of attack that a remote attack couldn't possibly carry out on this device is session hijacking. Session hijacking occurs when attackers take over a user session by obtaining session tokens or cookies, mitigated by implementing secure session handling practices like HTTPS-only cookies and session timeouts. DNS spoofing involves poisoning the DNS cache to redirect users to malicious websites, countered by DNSSEC (DNS Security Extensions) to ensure the integrity and authenticity of DNS responses. But since this device uses TLS, this makes the device secure against session hijacking.

Another possible attack to do is to use the well-known exploits of the earlier mentioned ports of the phone, especially the port 5353. For all vulnerabilities of this port can be accessed online.²

5 Analysis of weakness found

We have provided an analysis of the weaknesses found, presented in a table format below.

Weakness	Impact	Advice
Magnet replacement	Sensor does not detect that opening is open.	There is nothing you can really do against it, apart from having fast internet.
MDNS port 5353 exploits	Unauthorized Access, Denial of Service, Traffic amplification	Restrict traffic on UDP port 5353, regularly apply security patches.
DDoS	Disables the displayed state change of the APP	Only give people you trust access to your Wi-Fi and insert rate limiting
UUID in plain text in AP mode	Reconnaissance attacks	Only use AP mode if necessary.

6 Appendix

Below are various other screenshots of other Wireshark and Bettercap captures performed as part of our attacks against the device.

```
192.168.61.0/24 > 192.168.61.210 » set arp.spoof.targets 192.168.61.218
192.168.61.0/24 > 192.168.61.210 » arp.spoof on
192.168.61.0/24 > 192.168.61.210 » [13:21:40] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.61.0/24 > 192.168.61.210 » net.sniff on
192.168.61.0/24 > 192.168.61.210 »
```

Figure 44: ARP spoofing

```
192.168.61.0/24 > 192.168.61.210 » [13:25:52] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:26:49] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:26:51] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:27:41] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:27:53] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:28:02] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:33:34] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:33:54] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:34:09] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:34:26] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:35:30] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:35:51] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 » [13:36:03] [endpoint.new] endpoint 192.168.61.14 detected as 1c:90:ff:97:f0:dc.
192.168.61.0/24 > 192.168.61.210 » [13:36:22] [endpoint.lost] endpoint 192.168.61.14 1c:90:ff:97:f0:dc lost.
192.168.61.0/24 > 192.168.61.210 »
```

Figure 45: Endpoint detected when door sensor opened, but shortly after the connection gets lost

7 References

1) SpeedGuide (no date) Port 1035 (TCP/UDP), SpeedGuide.

Available at: <https://www.speedguide.net/port.php?port=1035> (Accessed: 29 February 2024).

2) SpeedGuide (no date b) Port 5353 (TCP/UDP), SpeedGuide.

Available at: <https://www.speedguide.net/port.php?port=5353> (Accessed: 29 February 2024).